

Performance Modeling for CNN Inference Accelerators on FPGA

Yufei Ma^{ID}, *Student Member, IEEE*, Yu Cao, *Fellow, IEEE*, Sarma Vrudhula^{ID}, *Fellow, IEEE*,
and Jae-Sun Seo, *Senior Member, IEEE*

Abstract—The recently reported successes of convolutional neural networks (CNNs) in many areas have generated wide interest in the development of field-programmable gate array (FPGA)-based accelerators. To achieve high performance and energy efficiency, an FPGA-based accelerator must fully utilize the limited computation resources and minimize the data communication and memory access, both of which are impacted and constrained by a variety of design parameters, e.g., the degree and dimension of parallelism, the size of on-chip buffers, the bandwidth of the external memory, and many more. The large design space of the accelerator makes it impractical to search for the optimal design in the implementation phase. To address this problem, a performance model is described to estimate the performance and resource utilization of an FPGA implementation. By this means, the performance bottleneck and design bound can be identified and the optimal design option can be explored early in the design phase. The proposed performance model is validated using a variety of CNN algorithms comparing the results with on-board test results on two different FPGAs.

Index Terms—Analytical modeling, convolutional neural networks (CNNs), field-programmable gate array (FPGA).

I. INTRODUCTION

MANY reported successes of convolutional neural networks (CNNs) for computer vision tasks [1]–[6] have motivated the development of hardware implementations of CNNs. In particular, there has been increased interest in field-programmable gate arrays (FPGAs) as a platform to accelerate the post-training inference computations of CNNs [7]–[13]. To achieve high performance and low energy cost, a CNN accelerator must: 1) fully utilize the limited computing resources to maximize the parallelism when executing the large number of operations for different convolution layers

with varying dimensions; 2) exploit the data locality by saving only the required data in on-chip buffers to minimize the cost of external memory (e.g., DRAM) accesses; and 3) manage the data storage patterns in buffers to increase the data reuse and reduce the data movements.

With the intervals of computation and off-chip communication overlapped using the dual buffering (or ping-pong buffering) technique, the performance of the CNN accelerator will be limited by either the computation delay or the DRAM transfer delay, and the actual bound will be determined by the values of the associated design parameters, as described by the *roofline model* in [7] and [9]. The computation delay is determined by the number of parallel processing engines (PEs), their utilization, and the operating frequency. The DRAM transfer latency is mainly affected by the external memory bandwidth and the number of DRAM accesses, and the latter is strongly affected by the size of the on-chip buffers. With regard to the energy efficiency (i.e., performance per watt), the main components that determine the dynamic power consumption are the computation logic and the memory traffic, the latter requiring efficient data movement and high data reuse. All these considerations show that there are numerous design parameters that determine the performance and energy efficiency of a CNN accelerator, making it impractical to find their optimal values during the implementation phase, as the synthesis of one FPGA design may take several hours. Robust and parametric models become a necessity for efficient design space exploration and selection of the optimal values of the design parameters. The architectural design space must be numerically characterized by design variables to control the accelerator performance and efficiency. For instance, loop optimization techniques [7], [11], such as loop unrolling and tiling, are employed to customize the acceleration strategy of parallel computation and data communication for convolution loops, whose variables in turn affect the resource utilization and memory access.

The starting point of this paper is a general system-level model of a CNN accelerator shown in Fig. 1, which includes the external memory, on-chip buffers, and PEs. The hardware architectural parameters, e.g., buffer sizes, are determined by the design variables that control the loop unrolling and tiling. Combining the design constraints and the choices of the acceleration strategy, a more fine-grained performance model is built to achieve better prediction for a specific design implementation, e.g., the design strategy in [11]. By this means,

Manuscript received August 27, 2018; revised December 6, 2018; accepted January 17, 2019. Date of publication February 4, 2019; date of current version March 18, 2020. This work was supported in part by the NSF I/UCRC Center for Embedded Systems through NSF under Grant 1230401, Grant 1237856, Grant 1701241, Grant 1361926, Grant 1535669, Grant 1652866, and Grant 1715443, in part by the Intel Labs, and in part by the Center for Brain-Inspired Computing (C-BRIC), one of six centers in JUMP, an SRC program sponsored by DARPA. This paper was recommended by Associate Editor Y. Wang. (Corresponding author: Yufei Ma.)

Y. Ma, Y. Cao, and J.-S. Seo are with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ 85287 USA (e-mail: yufeima@asu.edu; yu.cao@asu.edu; jaesun.seo@asu.edu).

S. Vrudhula is with the School of Computing, Informatics, Decision Systems Engineering, Arizona State University, Tempe, AZ 85287 USA (e-mail: vrudhula@asu.edu).

Digital Object Identifier 10.1109/TCAD.2019.2897634

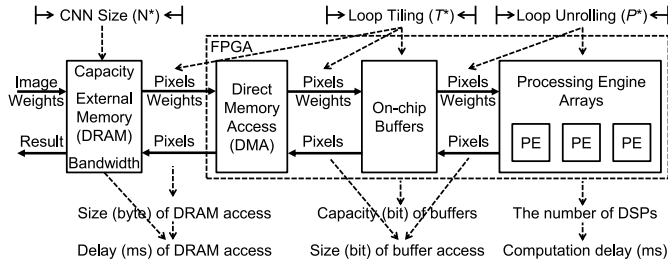


Fig. 1. General CNN hardware accelerator with three levels of hierarchy, where the loop design variables determine the key accelerator metrics, e.g., delay, resource usage, and memory access.

the proposed performance model makes it possible to identify the performance bottleneck and design limitations in the early development phase by exploring the design space through unrolling and tiling variables.

The main contributions of this paper are as follows.

- 1) The design objectives and resource costs are formulated using the design variables of loop unrolling and tiling.
- 2) A high-level performance model is proposed to estimate the accelerator throughput, on-chip buffer size, and the number of external and on-chip memory accesses.
- 3) The design space is efficiently explored through the proposed model instead of the real FPGA compilation to identify the performance bottleneck and obtain the optimal design configurations.
- 4) The performance model is validated for a specific design strategy across a variety of CNN algorithms comparing with the on-board test results on two different FPGAs.
- 5) Evaluate the techniques that may further enhance the performance of our current design by improving the efficiency of DRAM transactions and PE utilization.

The remainder of this paper is organized as follows. Section II overviews the procedure to map CNN operations onto an FPGA hardware system. A coarse-grained performance model is presented in Section III for rough estimation, and the fine-grained model is discussed in the following sections for a specific design strategy. Section IV estimates the size and latency of DRAM accesses. The latency of convolution and other layers are formulated and estimated in Section V. The on-chip buffer size requirement is analyzed in Section VI, and the size of buffer access is discussed in Section VII. Experiments are performed to explore the design space in Section VIII. Section IX evaluates the techniques that may further improve the current design performance.

II. CNN INFERENCE ACCELERATOR ON FPGA

A. Overview of Convolution Operation

The main operation in CNN algorithms involves accumulating the products of pixel values (e.g., features, activations, or neurons) with kernel weights, along different dimensions of the kernel and feature maps. Fig. 2 shows the four nested loops involved in CNNs. Note that the prefix N (for “number”) used in describing various parameters (e.g., N_{ix} , N_{iy} , N_{if} , etc.) denote the sizes of the kernel and feature maps. The

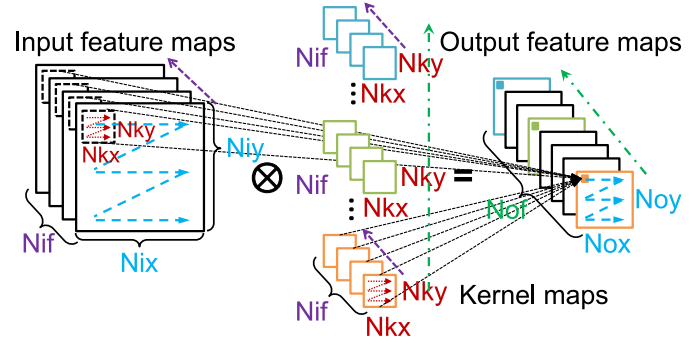


Fig. 2. Convolution operation is implemented by four levels of loops to MAC input features with kernel weights, where i : input, o : output, k : kernel, f : feature, x : x -axis (width), and y : y -axis (height), and the parameters of loop dimensions are prefixed with N [11].

loop operations shown in Fig. 2 are written as

$$\text{pixel}_L(\text{no}; x, y) = \sum_{ni=1}^{N_{if}} \sum_{ky=1}^{N_{ky}} \sum_{kx=1}^{N_{kx}} \text{pixel}_{L-1}(ni; S \times x + kx, S \times y + ky) \times \text{weight}(ni, \text{no}; kx, ky) + \text{bias}(\text{no}) \quad (1)$$

where S is the sliding stride, $x \in \{1, 2, \dots, N_{ox}\}$, $y \in \{1, 2, \dots, N_{oy}\}$, $\text{no} \in \{1, 2, \dots, N_{of}\}$, $L \in \{1, 2, \dots, \#\text{CONVs}\}$, and $\#\text{CONVs}$ is the number of convolution layers.

B. CNN Hardware Acceleration System

In the general model of a CNN accelerator shown in Fig. 1, due to the large data volume, both the weights and intermediate pixel results are stored in the external memory, e.g., DRAM. The input and weight on-chip buffers temporally store the input data to be processed by the PEs, and the PE results are saved in the output buffers. After completing the computation, the results are transferred back to DRAM from the output buffers, which will be used as the input to the subsequent layer.

C. Convolution Loop Optimization

Loop optimization techniques [7], [11], e.g., unrolling and tiling, are employed to customize the computation and communication patterns in a CNN accelerator. Loop unrolling directs the parallel computation along different convolution dimensions, and the variables representing the unrolling degrees are prefixed by P (see Fig. 3). These variables determine the number of PEs, which in turn determine the required number of DSPs in the FPGA to implement PEs, and thus decide the computation delay. The data flow from buffers into PEs is also impacted by loop unrolling variables, which affect the number of buffer access. Loop tiling divides a large CNN layer into multiple small tiles, which can be accommodated by on-chip buffers to increase data locality. Tiling sizes are represented by variables prefixed with T as shown in Fig. 3. The required buffer capacity is determined by the tiling variables, which also affect the DRAM access and thus the latency of DRAM transactions. The relationship between loop variables and the key specifications of accelerators, e.g., delay, DSP usage, buffer size, and memory access that affect memory power consumption, are shown in Fig. 1.

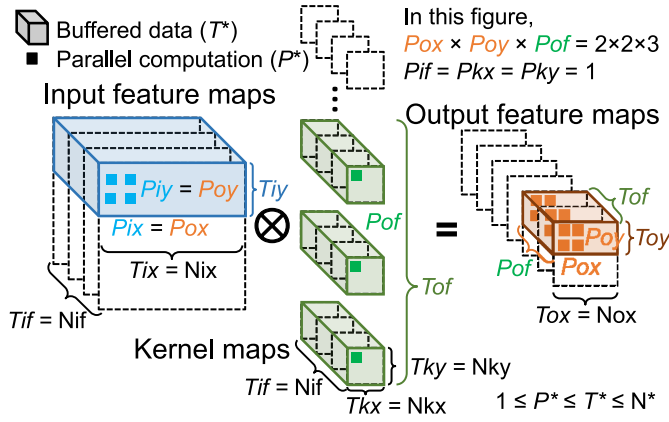


Fig. 3. Convolution acceleration strategy is customized by loop unrolling (P^*) for parallel computation and loop tiling (T^*) for data buffering. The parallelism is within one input feature map ($P_{ix} \times P_{iy}$) and across multiple kernel maps (P_{of}). The demanded buffer size can be changed by tuning variables T_{oy} and T_{of} [11].

D. Convolution Acceleration Strategy

To accurately predict the real implementation, a specific accelerator design strategy is needed to characterize the fine-grained performance model with detailed design options. The output stationary acceleration strategy of unrolling and tiling in [11] is adopted in this paper as shown in Fig. 3. The loop unrolling or the parallel computations are only performed within one input feature map ($P_{ix} = P_{ox} > 1$ and $P_{iy} = P_{oy} > 1$) and across multiple kernel maps ($P_{of} > 1$). That is, in Fig. 3, the $P_{ix} \times P_{iy}$ blue pixels in an input feature map are operated in parallel with green weights from P_{of} different kernel maps, resulting in $P_{ox} \times P_{oy}$ pixels in each of the P_{of} output feature maps. Therefore, the total number of PEs [multiply and accumulate (MAC) units] is $P_{ox} \times P_{oy} \times P_{of}$. The data required to compute one final output pixel are fully buffered, i.e., $T_{kx} = N_{kx}$, $T_{ky} = N_{ky}$, and $T_{if} = N_{if}$, so that the partial sum can be consumed inside the MAC unit without saving in the buffer. To ensure that the DRAM accesses are from continuous addresses, the entire row of the feature map is buffered, i.e., $T_{ix} = N_{ix}$ and $T_{ox} = N_{ox}$. If the on-chip RAM capacity is large enough, either all pixels or all weights of one layer are fully buffered, so that each data only needs to be fetched from DRAM once to reduce DRAM access. Finally, the required buffer sizes of each layer can be changed by tuning T_{oy} and T_{of} .

III. COARSE-GRAINED PERFORMANCE MODEL

In this section, a coarse-grained performance model of a general CNN accelerator that is independent of a specific acceleration strategy, is presented. Then, more detailed design choices and constraints (e.g., unrolling and tiling variable settings, memory storage pattern, and computation dataflow) are introduced to create a more precise and fine-grained model in the following sections. Table I lists the mainly used abbreviations and units in this paper, which indicate the meaning of the variables discussed afterward.

TABLE I
LIST OF ABBREVIATIONS AND UNITS

Abbreviation	Description	Abbreviation	Description
P_x	Pixel	R_d	Read
W_t	Weight	W_r	Write
B_{uf}	Buffer	InB_{uf}	Input Buffer
WtB_{uf}	Weight Buffer	$OutB_{uf}$	Output Buffer
BW	Bandwidth	$1T$	One Tile
Unit	Description	Unit	Description
$bit / byte$	Data Size	$word$	RAM Depth
ms	Delay Time	MHz	Frequency

A. Computation Latency

The number of multiplication operations per layer is $N_m = N_{if} \times N_{kx} \times N_{ky} \times N_{of} \times N_{ox} \times N_{oy}$. The number of PEs that determines the degree of parallel computations by unrolling is $P_m = P_{if} \times P_{kx} \times P_{ky} \times P_{of} \times P_{ox} \times P_{oy}$. A similar reasoning is applied to determine the number of clock cycles for one buffered tile ($1T$) of convolution. This is denoted by $\#cycles_{1T}$, and is expressed as follows:

$$\#cycles_{1T} = \left\lceil \frac{T_{if}}{P_{if}} \right\rceil \left\lceil \frac{T_{kx}}{P_{kx}} \right\rceil \left\lceil \frac{T_{ky}}{P_{ky}} \right\rceil \left\lceil \frac{T_{of}}{P_{of}} \right\rceil \left\lceil \frac{T_{ox}}{P_{ox}} \right\rceil \left\lceil \frac{T_{oy}}{P_{oy}} \right\rceil. \quad (2)$$

The number of tiles for one convolution layer is

$$\#tiles = \left\lceil \frac{N_{if}}{T_{if}} \right\rceil \left\lceil \frac{N_{kx}}{T_{kx}} \right\rceil \left\lceil \frac{N_{ky}}{T_{ky}} \right\rceil \left\lceil \frac{N_{of}}{T_{of}} \right\rceil \left\lceil \frac{N_{ox}}{T_{ox}} \right\rceil \left\lceil \frac{N_{oy}}{T_{oy}} \right\rceil. \quad (3)$$

The total number of computation clock cycles of one convolution (CV) layer is

$$\#cycles_{1CV} = \#tiles \times \#cycles_{1T}. \quad (4)$$

B. On-Chip Buffer Size

Determined by the tiling variables, the input buffer (InB_{uf}) size (bit) requirement to store one tile of input pixels is

$$bit_{InB_{uf}} = T_{ix} \times T_{iy} \times T_{if} \times bit_{Px} \quad (5)$$

where bit_{Px} is the bit width of one pixel (P_x). Similarly, the size (bit) requirement of weight buffer (WtB_{uf}) to store one tile of weights is

$$bit_{WtB_{uf}} = T_{kx} \times T_{ky} \times T_{if} \times T_{of} \times bit_{Wt} \quad (6)$$

where bit_{Wt} is the bit width of one weight (W_t). The output buffer ($OutB_{uf}$) size (bit) requirement to store one tile of output pixels is

$$bit_{OutB_{uf}} = T_{ox} \times T_{oy} \times T_{of} \times bit_{Px}. \quad (7)$$

The theoretical sizes of the input, weight, and output buffers are the maximum possible values of $bit_{InB_{uf}}$, $bit_{WtB_{uf}}$, and $bit_{OutB_{uf}}$ of all the convolution layers, respectively. In an actual implementation, the sizes of the buffers used may be larger than these values due to inefficient storage pattern and extra garbage data.

C. DRAM Access and Latency

In theory, the size of one tile of data read from or written to the external DRAM should be the same as the size of buffered data. Therefore, the size (bytes) of input pixels (Px) read (Rd) from DRAM for one convolution tile is $byte_RdPx = bit_InBuf/8$. The size (bytes) of one tile of weights (Wt) read from the DRAM is $byte_RdWt = bit_WtBuf/8$. The size (bytes) of one tile of output pixels written (Wr) to the DRAM is $byte_WrPx = bit_OutBuf/8$. The latency (milliseconds or ms) of DRAM transactions of one tile (1T) of data is determined by the size of DRAM access and the memory bandwidth. This is given by

$$ms_DRAM_1T = \frac{byte_DRAM_1T}{BW_Memory \times 10^6} \quad (8)$$

where BW_Memory is the external memory bandwidth (GB/s) and $byte_DRAM_1T$ is the size of DRAM access of one tile, which can be $byte_RdPx$, $byte_RdWt$, or $byte_WrPx$.

D. On-Chip Buffer Access

The size (bits) of on-chip buffer access (bit_Buf_Access) is computed by multiplying the number of access clock cycles ($\#cycles_Access$) with the total bit width of the corresponding buffers ($width_Buf$)

$$bit_Buf_Access = \#cycles_Access \times width_Buf. \quad (9)$$

During computation, it is assumed that data are continuously read from input and weight buffers and the results are written into the output buffers every clock cycle. Then, to estimate the buffer access during computation, $\#cycles_Access$ equals the number of computation cycles, and $width_Buf$ can be the total bit width of input/weight/output buffers. The size (bits) of buffer access by direct memory access (DMA) that writes into input and weight buffers and reads from output buffers is the same as the size of external memory access. The data stored in the input or weight buffers may be read multiple times during computation, hence the size of data read from buffers may be larger than the size of data written into buffers from DRAM. Since each result is written into output buffers only once, the size of write and read operations of output buffers is the same.

E. Other Implementation Methods of Convolution

Instead of the aforementioned direct implementation of the convolution loop operations, convolution can also be performed as matrix multiplication [8] or accelerated in the frequency domain [14], [15]. Since these methods require significantly different hardware architecture and dataflow, we only briefly analyze them with our modeling parameters.

1) *Matrix Multiplication*: The MAC operations in convolution can be mapped to matrix multiplication [8], which can utilize the library optimized for GPU, e.g., BLAS used by Caffe. The original 4-D kernel weights are transformed to be a matrix with Nof rows and $Nkx \cdot Nky \cdot Nif$ columns. The 3-D input feature map is transformed into a matrix with $Nkx \cdot Nky \cdot Nif$ rows and $Nox \cdot Noy$ columns. There are redundant data in the transformed feature matrix due to

the overlapped sliding of the kernel window. Therefore, this method could lead to either complex dataflow and extra hardware to perform the transform on the fly or additional DRAM accesses due to the redundant data.

2) *Fast Fourier Transform*: FFT [16] can reduce the number of multiplications from $\Theta(Nox \cdot Noy \cdot Nkx \cdot Nky)$ to $\Theta(Nox \cdot Noy \cdot \log_2(Nox))$ and even further to $\Theta(Nox \cdot Noy \cdot \log_2(Nkx))$ with Overlap-and-Add [16]. The original kernel weights and input features are transformed into the frequency domain to do multiplications, and then the inverse FFT is applied to recover the results. Therefore, extra hardware is required to implement the transform. In addition, the computation reduction is decreased with smaller kernel size.

3) *Winograd Transform*: Winograd transform [15] is another approach to reduce the convolution operations. The number of multiplications of the original convolution in one tile is $\Theta((Ttx \cdot Tox)(Tky \cdot Toy))$, while in Winograd it is only $\Theta((Ttx + Tox - 1)(Tky + Toy - 1))$. For example, with kernel size of 3×3 and tiled output features of 2×2 , we can achieve $2.25\times$ reduction of multiplication operations. However, the addition operations are increased in Winograd, and additional storage and bandwidth are required by the transform matrices. The operations can be further reduced with larger feature tiles, but the complexity of the transform matrix will significantly increase. Since Winograd essentially unrolls the computation within a kernel window, the varying kernel sizes can affect its computation efficiency.

IV. MODELING OF DRAM ACCESS

In this section, more accurate models of the DRAM access are constructed by including the design constraints and the variables of loop acceleration described in Section II-D.

A. Data Size of Convolution DRAM Access

The DMA engine shown in Fig. 1 is used to transfer data to and from off-chip DRAM. To achieve the maximum bandwidth, the data width of both the DMA (bit_DMA) and the DRAM controller (bit_DRAM) are set to be 512 bits.

Pox represents the number of pixels that are computed in parallel in each output feature map. For the feature map transfer, the number of groups of Pox pixels associated with one DMA address is then given by $\#PoxGroup = \lfloor bit_DMA / (Pox \times bit_Px) \rfloor$, where bit_Px is the bit width per pixel. The effective or actual DMA bandwidth (as a fraction of the maximum) is then given by

$$eff_DMA_Px = \frac{\#PoxGroup \times Pox \times bit_Px}{bit_DMA}. \quad (10)$$

For example, if $Pox = 7$, $bit_DMA = 512$, and $bit_Px = 16$, then there are $\#PoxGroup = 4$ groups of Pox pixels in one DMA address, and $4 \times 7 \times 16 = 448$ bits are the effective number of bits out of the DMA bit width of 512 bits, resulting in $eff_DMA_Px = 0.875$.

The intermediate pixel results stored in DRAM are arranged row-by-row, map-by-map, and layer-by-layer. One convolution tile needs $Tix \times Tiy \times Tif$ input pixels. Then, the size (bytes)

of the input pixels read (Rd) from the DRAM for one tile is

$$byte_RdPx = \frac{Tix \times Tiy \times Tif \times bit_Px}{eff_DMA_Px \times 8}. \quad (11)$$

Note that if $eff_DMA_Px < 1$, it implies more bytes are read than necessary due to the alignment of data storage. Similarly, the size (bytes) of output pixels written (Wr) to DRAM for one convolution tile is

$$byte_WrPx = \frac{Tox \times Toy \times Tof \times bit_Px}{eff_DMA_Px \times 8}. \quad (12)$$

For convolution weights, the ratio of effective DRAM bandwidth to the maximum of reading weights from DRAM is

$$eff_DMA_Wt = \frac{\lfloor bit_DMA/bit_Wt \rfloor \times bit_Wt}{bit_DMA}. \quad (13)$$

The size (bytes) of input weights read from DRAM for one convolution tile is

$$byte_RdWt = \frac{Tkx \cdot Tky \cdot Tif \cdot Tof \cdot bit_Wt}{eff_DMA_Wt \times 8}. \quad (14)$$

B. DRAM Access Delay of One Tile

The data width of the DRAM controller interface to the FPGA is assumed to be bit_DRAM , running at frequency of MHz_DRAM . This means the theoretical maximum DRAM bandwidth (BW_DRAM in GB/s) is $(bit_DRAM/8) \times (MHz_DRAM/10^3)$, which is normally very difficult to sustain due to the noncontiguous DRAM access. For example, if $bit_DRAM = 512$ bits, with $MHz_DRAM = 266$ MHz, then $BW_DRAM = (512/8) \times (266/10^3) = 17.0$ GB/s as the maximum DRAM bandwidth.

In the CNN acceleration system described in [11], the DMA engine is operated at the same clock frequency as the CNN accelerator core (i.e., $MHz_Accelerator$) with read/write data width (bit_DMA) of 512 bits. An asynchronous first-input, first-output (FIFO) can be inserted between DMA and the DRAM controller to synchronize data across the two clock domains. Then, the DMA bandwidth (BW_DMA) is $(bit_DMA/8) \times (MHz_Accelerator/10^3)$. By this means, the bandwidth of the external memory is bounded by the effective bandwidth of both the DRAM controller and the DMA as $BW_Memory = \min(BW_DRAM, BW_DMA)$, which is used in (8) to calculate the DRAM latency.

The more accurate and specific DRAM access sizes of one tile ($byte_DRAM_1T$) are discussed in this section, including $byte_RdPx$, $byte_WrPx$, and $byte_RdWt$. Then, we can use (8) to compute their corresponding DRAM access delay (ms_DRAM_1T), e.g., ms_RdPx , ms_WrPx , and ms_RdWt , respectively.

C. DRAM Access of Other Layers

The DRAM access and performance of other layers, e.g., max pooling, fully connected (FC), and Eltwise, are also investigated and included in our performance model. Since the analysis process of these layers are similar to the convolution layer, for simplicity, their detailed formulas used in the performance model are not presented.

The pixels of max-pooling layers are also transferred to and from the DRAM with loop tiling performed, depending on the adopted design choices [11], [17]. For max pooling, the calculation of the DRAM transfer sizes of input and output pixels are similar to $byte_RdPx$ in (11) and $byte_WrPx$ in (12), respectively.

The weights of FC layers are stored in DRAM in the same way as convolution and reuse the same weight buffers. Since the intermediate results of FC layers are small (< 20 KB), they are always kept in the on-chip RAMs.

The Eltwise layer performs element-wise summation of the outputs of two layers. The Eltwise layer is executed after its two precedent layers are finished, so that it can directly read the results of one layer from the output buffers, without accessing DRAM. However, the Eltwise layer still needs to read the outputs of the other layer from DRAM as the output buffers were already refreshed.

V. MODELING OF LATENCY

A. Computation Delay (ms) of One Convolution Tile

Setting $Pif = Pkx = Pky = 1$, $Tif = Nif$, $Tkx = Nky$, $Tkx = Nky$, and $Tox = Nox$ as described in Section II-D, (2) can be written as

$$\#cycles_1T = Nif \cdot Nkx \cdot Nky \cdot \left\lceil \frac{Tof}{Pof} \right\rceil \cdot \left\lceil \frac{Nox}{Pox} \right\rceil \cdot \left\lceil \frac{Toy}{Poy} \right\rceil. \quad (15)$$

Then, the computation delay (ms) of one convolution tile is

$$ms_Compute = \frac{\#cycles_1T}{MHz_Accelerator \times 10^3} \quad (16)$$

where $MHz_Accelerator$ is the clock frequency of the accelerator in MHz. The number of tiles of one convolution layer ($\#tiles$) is $\lceil Nof/Tof \rceil \lceil Noy/Toy \rceil$ based on (3) with $Nif = Tif$, $Nkx = Tkx$, $Nky = Tky$, and $Nox = Tox$ as described in Section II-D.

B. Overall Delay (ms) of One Convolution Layer

With dual buffering technique, the DRAM access is overlapped with computation to improve performance [7], [10]. The overall tile-by-tile delay of one convolution layer is illustrated in Fig. 4. Since the dual buffering pipeline is only within one layer with the current design choice, after the start of one layer and before the computation of the first tile, both the input pixels and weights (Wt) of one tile are first read from DRAM. This is shown as “Input + Wt ” at the beginning of one layer in Fig. 4. Similarly, after the completion of the last tile’s computation, its output pixels are transferred back into DRAM, which is shown as “Output” at the end in Fig. 4. Therefore, for each convolution layer, the delay of transferring inputs of the first tile and outputs of the last tile cannot be overlapped with the computation, and this delay is denoted as

$$ms_Mem = ms_RdPx + ms_RdWt + ms_WrPx. \quad (17)$$

If the convolution layer has only one tile that is $Tiy = Niy$ and $Tof = Nof$, there is no overlapping of memory transfer and computation as shown in Fig. 4(a), and the delay of this tile

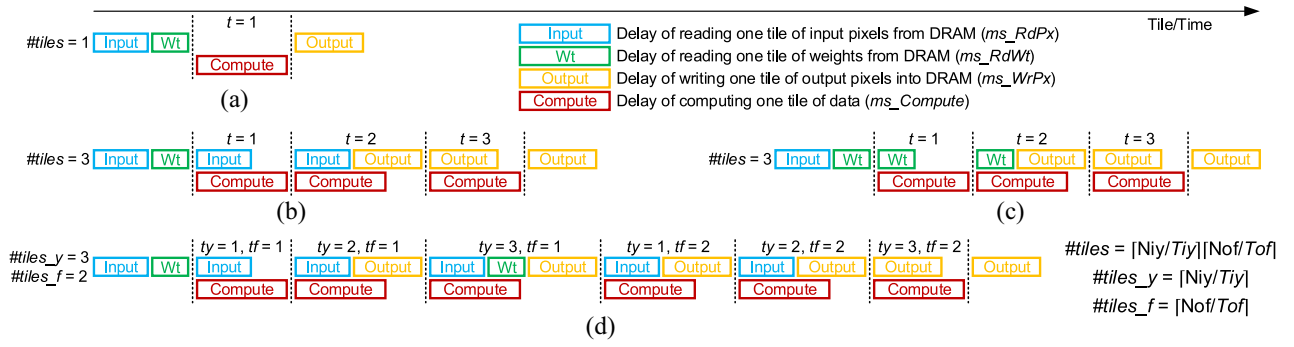


Fig. 4. Tile-by-tile delay of one convolution layer, and the DRAM access delay is overlapped with the computation delay due to dual buffering technique. (a) Both inputs and weights fully buffered, (b) only weights fully buffered, (c) only inputs fully buffered, and (d) neither inputs nor weights fully buffered.

Algorithm 1: Delay Estimation of one Convolution Layer (ms_ICV), Where $C = ms_Compute$, $I = ms_RdPx$, $W = ms_RdWt$, and $O = ms_WrPx$

```

input :  $C, I, W, O, \#tiles, \#tiles_y, \#tiles_f$ 
output:  $ms\_ICV$ 
1 if  $Tiy = Niy$  and  $Tof = Nof$  then
2    $T[1] = C$ 
3 else if  $Tiy < Niy$  and  $Tof = Nof$  then
4   for  $t = 1$  to  $\#tiles$  do
5     if  $t = 1$  then
6        $T[t] = \max(C, I)$ 
7     else if  $t = \#tiles$  then
8        $T[t] = \max(C, O)$ 
9     else
10       $T[t] = \max(C, I + O)$ 
11    end
12  end
13 else if  $Tiy = Niy$  and  $Tof < Nof$  then
14  for  $t = 1$  to  $\#tiles$  do
15    if  $t = 1$  then
16       $T[t] = \max(C, W)$ 
17    else if  $t = \#tiles$  then
18       $T[t] = \max(C, O)$ 
19    else
20       $T[t] = \max(C, W + O)$ 
21    end
22  end
23 else
24  for  $tf = 1$  to  $\#tiles_f$  do
25    for  $ty = 1$  to  $\#tiles_y$  do
26       $t = ty + (tf - 1) \times \#tiles_y$ ;
27      if  $ty = 1$  and  $tf = 1$  then
28         $T[t] = \max(C, I)$ 
29      else if  $t = \#tiles$  then
30         $T[t] = \max(C, O)$ 
31      else if  $ty = \#tiles_y$  then
32         $T[t] = \max(C, I + W + O)$ 
33      else
34         $T[t] = \max(C, I + O)$ 
35      end
36    end
37  end
38 end
39  $ms\_ICV = \sum_{t=1}^{\#tiles} T[t] + ms\_Mem$ 

```

[e.g., $t = 1$ in Fig. 4(a)] is only determined by the computation delay as in Algorithm 1 (line 2).

If the convolution layer has multiple tiles and all its weights are fully buffered, i.e., $Tiy < Niy$ and $Tof = Nof$, then the

weights only need to be read from DRAM once and can be reused by different tiles as illustrated in Fig. 4(b). The procedure to estimate the delay of this convolution layer is summarized in Algorithm 1 (lines 3–12). The computation of the first tile [e.g., $t = 1$ in Fig. 4(b)] is overlapped with fetching the input pixels of the next tile, and there is no DMA transfer of output pixels of the previous layer, thus the delay of this tile is determined by Algorithm 1 (line 6). The computation of the last tile [e.g., $t = 3$ in Fig. 4(b)] is overlapped with transferring the output pixels of its previous tile, and its delay is calculated by Algorithm 1 (line 8). For the other tiles [e.g., $t = 2$ in Fig. 4(b)], the communication with DRAM includes both reading input pixels and writing output pixels, and the delay of one tile is expressed by Algorithm 1 (line 10). The overall delay of this convolution layer is the sum of all the tiles as well as the DRAM access delay before the first tile and after the last tile, i.e., ms_Mem .

If the convolution layer has multiple tiles and all its pixels are fully buffered, i.e., $Tiy = Niy$ and $Tof < Nof$, then the pixels only need to be read from DRAM once and can be reused by different tiles as illustrated in Fig. 4(c). Similarly, the procedure to estimate the delay of this convolution layer is summarized in Algorithm 1 (lines 13–22).

If neither the weights nor the pixels of the convolution layer can be fully buffered, i.e., $Tiy < Niy$ and $Tof < Nof$, its pipeline schedule is shown in Fig. 4(d) and the associated delay is estimated in Algorithm 1 (lines 23–37). In this case, either the pixels or the weights need to be refetched multiple times from the DRAM. In our current design, the input pixels are refetched and the weights only need to be read once. If the DRAM access requirement of input pixels is more than weights, we can also refetch weights instead and only read input pixels once by changing the DMA instructions and associated control logic. Before the computation, the first tile of weights is loaded and reused by the following consecutive $\#tiles_y = \lceil Niy/Tiy \rceil$ tiles of pixels to perform convolution. Then, the next tile of weights is loaded and reused by the following $\#tiles_y$ tiles of pixels. This process iterates by $\#tiles_f = \lceil Nof/Tof \rceil$ times to complete the computation with all the $\#tiles_f$ tiles of weights. By this means, the pixels are refetched by $\#tiles_f$ times. A normal tile needs to read input pixels of the next tile from DRAM and write output pixels of the previous tile into DRAM, where the required weights are already loaded during the previous tile and reused. Therefore, the delay of a normal tile is estimated as in Algorithm 1

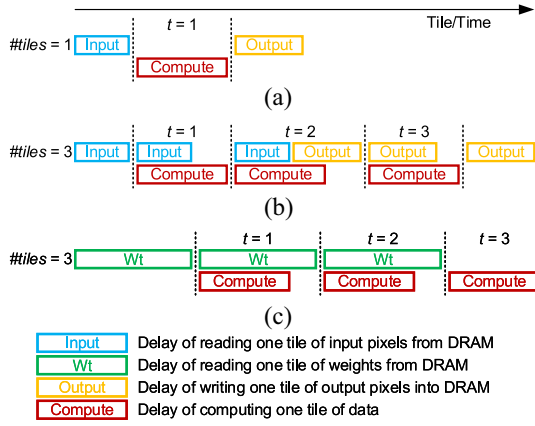


Fig. 5. Tile-by-tile delay of one pooling/FC layer, and the DRAM access delay is overlapped with the computation delay due to the dual buffering technique. (a) Max pooling: inputs fully buffered in one tile. (b) Max pooling: inputs partially buffered in multiple tiles. (c) FC.

(line 34). As the first tile does not have a previous tile, there is no transfer of output pixels back to DRAM as in Algorithm 1 (line 28). For the last tile, there is no need to read input pixels for the next tile as in Algorithm 1 (line 30). When $\#tiles_y$ tiles of weights are finished [e.g., $ty = 3$ and $tf = 1$ in Fig. 4(d)], new tile of weights is loaded from DRAM, and DRAM access also includes transfer of pixels as in Algorithm 1 (line 32).

C. Delay Estimation of Other Layers

With the dual buffering technique employed, the overall tile-by-tile process of one max-pooling layer is illustrated in Fig. 5(a) and (b), which is similar to the convolution layer except that pooling does not need weights. If the pooling layer has only one tile, which means the inputs of one pooling layer can be fully buffered, there is no overlapping between memory transfer and computation as shown in Fig. 5(a) and (b) illustrates the dual buffering pipeline of one pooling layer with multiple tiles. Similar to Algorithm 1, we can compute the overall latency of max-pooling layers according to the tile-by-tile execution schedule, with the delay of max-pooling computation and DRAM access calculated similar to the convolution layer.

Fig. 5(c) shows the pipeline schedule of FC layer, where weights are fetched before the corresponding computation and no outputs are transferred back to DRAM. The storage format of FC weights in the weight buffer allows us to read Pof weights simultaneously every clock cycle to parallel compute Pof outputs. Then, the computation cycles of one FC tile equal to the depth of buffered FC weights. The overall delay of FC is bounded and determined by the computation delay or the DRAM access delay of weights.

VI. SIZE REQUIREMENT OF ON-CHIP MEMORY

With the specific data storage pattern of buffers, we can more precisely calculate the required on-chip buffer sizes than the rough estimation in Section III-B.

$r(i, y)$ is y -th row in i -th input feature map, $i \in \{1, 2, \dots, Tif\}$, $y \in \{1, 2, \dots, Tiy - 2 \times \text{padding}\}$
 $c(x)$ is the x -th column element in one row, $x \in \{1, 2, \dots, Tix - 2 \times \text{padding}\}$

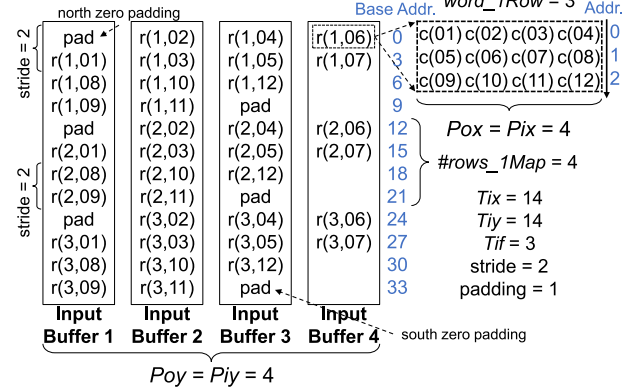


Fig. 6. Convolution data storage pattern in the input pixel buffers.

A. Size and Storage of Input Buffers

Fig. 6 illustrates the proposed storage pattern of convolution input pixels, which benefits the dataflow of $Pox \times Poy$ pixels from buffers into MAC units [11]. The width of one input buffer is determined by Pox to feed data for parallel computation of Pox pixels in one feature map row. The number of input buffers is determined by Poy to feed data for parallel computation of Poy multiple output rows. In Fig. 6, $c(x)$ denotes one input pixel in the x th column of a certain row, where $x \in \{1, 2, \dots, Tix - 2 \times \text{padding}\}$ and Tix includes both the east and west zero padding. The east and west zero paddings are not stored in buffers and instead they are masked out by control logic before loading into the MAC units. The number of addresses or words occupied by one row is

$$word_1Row = \lceil (Tix - 2 \times \text{padding}) / Pox \rceil. \quad (18)$$

In Fig. 6, $r(i, y)$ is the y th row of the i th input feature map, where $i \in \{1, 2, \dots, Tif\}$ and $y \in \{1, 2, \dots, Tiy\}$. The Tiy rows of one input feature map, including north and south zero paddings if they exist, are distributed across the Poy number of input buffers. With $stride = 2$ as in Fig. 6, two adjacent rows are continuously stored in the same buffer according to the dataflow requirement. Then, the number of rows of one feature map, i.e., $r(i, y)$, in one buffer is

$$\#rows_1Map = \lceil \lceil Tiy / stride \rceil / Poy \rceil \times stride. \quad (19)$$

The storage location of the subsequent input feature maps is aligned with the first feature map to simplify the address generation logic, which causes some overhead due to the non-continuous storage pattern as shown by the blank spaces in the buffers in Fig. 6. By this means, the depth or words requirement of one input buffer ($InBuf$) storing Tif input feature maps for one convolution layer is expressed as

$$word_InBuf = word_1Row \cdot \#rows_1Map \cdot Tif. \quad (20)$$

The data width of one input buffer is $Pox \times bit_Px$ and the number of input buffers is $Poy \times Dual$ with $Dual = 2$, where $Dual$ represents doubling of the number of buffers due to the dual buffer structure. Therefore, in every clock cycle, $Pox \times Poy$

$w(i,o)$ is one kernel window of i -th input channel and o -th output channel, $i \in \{1, 2, \dots, Tif\}$, $o \in \{1, 2, \dots, Tof\}$

$k(x,y)$ is one kernel weight inside the kernel window, $x \in \{1, 2, \dots, Ttx\}$, $y \in \{1, 2, \dots, Tky\}$

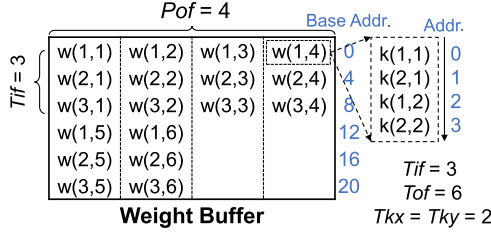


Fig. 7. Convolution data storage pattern in the weight buffer.

pixels can be fed into the MAC units. The input buffer size requirement of one convolution layer is

$$bit_InBuf = Dual \times Poy \times Pox \times bit_Px \times word_InBuf. \quad (21)$$

The final input buffer size is the maximum bit_InBuf of all the convolution layers. The actual input buffer size in (21) is larger than the rough estimation in (5) due to the mismatch of tile and buffer dimensions caused by the specific storage pattern.

B. Size and Storage of Weight Buffers

The storage pattern of weight buffer is illustrated in Fig. 7. The $k(x,y)$ in Fig. 7 denotes one weight inside the $Nkx \times Nky$ kernel window, where $x \in \{1, 2, \dots, Ttx\}$ and $y \in \{1, 2, \dots, Tky\}$. In the chosen design, we always have $Ttx = Nkx$ and $Tky = Nky$, so that one kernel window is fully buffered. These $Ttx \times Tky$ weights, i.e., $k(x,y)$, are stored in continuous addresses as we serially compute one kernel window, e.g., $Pkx = Pky = 1$. In Fig. 7, $w(i,o)$ denotes one kernel window of the i th input channel and o th output channel, which is comprised of $Ttx \times Tky$ weights. Weights from different input channels (Tif) are stacked in different addresses as we serially compute each input channel. To compute Pof output channels in parallel, the weights of Pof output channels are stored at the same address of the weight buffer. Therefore, the bit width of the weight buffer is $Pof \times bit_Wt$. The words or depth of the weight buffer ($WtBuf$) is

$$word_WtBuf = Ttx \times Tky \times Tif \times \lceil Tof/Pof \rceil. \quad (22)$$

With dual buffering, the number of weight buffers is two. The weight buffer size requirement of one convolution layer is

$$bit_WtBuf = Dual \cdot Pof \cdot bit_Wt \cdot word_WtBuf. \quad (23)$$

If Tof/Pof is not an integer, some blank spaces in the weight buffer are wasted as in Fig. 7. The final weight buffer size is the maximum bit_WtBuf of all the convolution layers.

C. Size and Storage of Output Buffers

After every $Nkx \times Nky \times Nif$ clock cycles, there are $Pox \times Poy \times Pof$ outputs from MAC units. To reduce the bit width of data bus and the bandwidth requirement of output

$r(o,y)$ is y -th row in o -th output feature map, $o \in \{1, 2, \dots, Tof\}$, $y \in \{1, 2, \dots, Toy\}$
 $c(x)$ is the x -th column element in one row, $x \in \{1, 2, \dots, Tox\}$

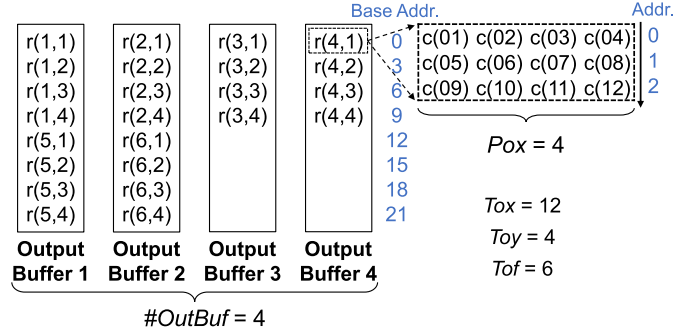


Fig. 8. Convolution data storage pattern in the output pixel buffers.

buffers as in Fig. 8, the parallel outputs are serialized into $Poy \times \lceil Pof/\#OutBuf \rceil$ clock cycles, where $\#OutBuf$ is the number of output buffers excluding the dual buffer structure with $\#OutBuf \leq Pof$. By this means, the data width of one output buffer is $Pox \times bit_Px$, as shown in Fig. 8, to store the parallel Pox outputs from the same feature map.

The output buffer storage pattern is illustrated in Fig. 8, where $c(x)$ is the x th column element in one row with $x \in \{1, 2, \dots, Tox\}$ and $r(o,y)$ is the y th row in the o th output feature map with $o \in \{1, 2, \dots, Tof\}$ and $y \in \{1, 2, \dots, Toy\}$. The outputs of the same feature map are continuously stored in the same buffer in a row-major order. One row ($r(o,y)$) is comprised of Tox elements ($c(x)$) continuously stored in $\lceil Tox/Pox \rceil$ addresses, and we set $Tox = Nox$ so that one entire row is processed while maintaining the row-major order. One feature map has Toy number of rows stored in one buffer and it occupies $Toy \times \lceil Tox/Pox \rceil$ addresses. One output buffer stores $\lceil Tof/\#OutBuf \rceil$ number of feature maps. Then, the number of words or the depth of one output buffer ($OutBuf$) for one convolution layer is

$$word_OutBuf = \lceil Tof/\#OutBuf \rceil Toy \lceil Tox/Pox \rceil. \quad (24)$$

The output buffer size requirement of one convolution layer is

$$bit_OutBuf = (Dual \times \#OutBuf) \times (Pox \times bit_Px) \times word_OutBuf. \quad (25)$$

If $Tof/\#OutBuf$ is not an integer, the blank spaces in the output buffers as in Fig. 8 are wasted.

D. Size and Storage of Pooling Buffers

The max-pooling layers share the input and output buffers with convolution layers. Due to the different dataflow requirement, the max-pooling input storage pattern in the input buffers is different from convolution inputs, but it is the same as the output storage pattern of convolution outputs in Fig. 8. In addition, the output buffer storage pattern of max-pooling layers is also the same as the convolution outputs in Fig. 8. The pixels from the same feature map are stored in the same buffer, and different feature maps are distributed across different buffers. Therefore, the input and output buffer depth of one tile of max pooling is similar to (24). The buffer size requirement of pooling layers is ensured to be smaller than

that of the convolution layers by using smaller pooling tiling variables so that there is no overflow of pooling data.

VII. MODELING OF ON-CHIP BUFFER ACCESS

The energy cost of accessing data in the buffers dominates the on-chip memory energy consumption [18], [19], so it is essential to reduce the size of buffer accesses for energy-efficient design. To reduce the buffer access size, data should be reused as much as possible either by multiple PEs or by different execution tiles, which will be discussed in this section.

A. Reading Input and Weight Buffers of Convolution

Based on (9) to estimate the buffer access, we need to compute $\#cycles_Access$ first. In this case, $\#cycles_Access$ is the MAC computation clock cycles of one tile, which is $\#cycles_1T$ in (15). Then, the computation clock cycles of all the convolution layers are

$$\#cycles_C = \sum_{L=1}^{\#CONVs} \#cycles_1T[L] \times \#tiles[L] \quad (26)$$

where $\#CONVs$ is the number of convolution layers and $\#tiles$ is the number of tiles. The size (bit) of data read (Rd) from input buffers ($InBuf$) for convolution layers is computed by multiplying the read clock cycles with the total input buffer data width as

$$bit_RdInBuf = \#cycles_C \cdot (Pox \cdot Poy \cdot bit_Px) \quad (27)$$

where every $Pox \times Poy$ pixels are reused by Pof MAC units and the number of input buffer accesses is reduced by Pof times. Similarly, the size (bit) of data read (Rd) from weight buffers ($WtBuf$) for all the convolution layers is

$$bit_RdWtBuf = \#cycles_C \times (Pof \times bit_Wt) \quad (28)$$

where every Pof weights are reused by $Pox \times Poy$ MAC units and the number of weight buffer accesses is reduced by $Pox \times Poy$ times.

B. Writing Input and Weight Buffers of Convolution

Before computation, the input data are written into the input and weight buffers from DMA. As discussed in Section V-B, not every tile needs to read both pixels and weights from DRAM, because some pixels or weights of one tile can be reused by the following adjacent tiles. The number of tiles of one convolution layer that write new weights (Wt) to the weight buffer is

$$\#tiles_Wt = \lceil Nof / Tof \rceil. \quad (29)$$

The number of tiles of one convolution layer that write new input pixels (In) to the input buffers is

$$\#tiles_In = \begin{cases} \lceil \frac{Noy}{Toy} \rceil \lceil \frac{Nof}{Tof} \rceil, & \text{if } Toy < Noy \text{ and } Tof < Nof \\ \lceil \frac{Noy}{Toy} \rceil, & \text{otherwise.} \end{cases} \quad (30)$$

When neither weights nor pixels are fully buffered, i.e., $Toy < Noy$ and $Tof < Nof$, the same pixels are reloaded $\lceil Nof / Tof \rceil$

times into input buffers as shown in Fig. 4(d). Similar to (21), the size (bit) of one tile ($1T$) of pixels written into the input buffers is

$$bit_WrIn_1T = word_InBuf \cdot Poy \cdot Pox \cdot bit_Px. \quad (31)$$

The size (bit) of data loaded into the input buffers of all the convolution layers is

$$bit_WrInBuf = \sum_{L=1}^{\#CONVs} bit_WrIn_1T[L] \times \#tiles_In[L]. \quad (32)$$

Similarly, the size (bit) of one tile of weights written into the weight buffers is

$$bit_WrWt_1T = word_WtBuf \times Pof \times bit_Wt \quad (33)$$

and the size (bit) of data written into the weight buffers of all the convolution layers is

$$bit_WrWtBuf = \sum_{L=1}^{\#CONVs} bit_WrWt_1T[L] \times \#tiles_Wt[L]. \quad (34)$$

C. Data Access of Output Buffers of Convolution

The number of clock cycles to write outputs into output buffers during one tile is the same as $word_OutBuf$, where one word of data is written into one output buffer in one cycle. Since every tile of one layer has outputs to be saved, the clock cycles of writing outputs to output buffers are $word_OutBuf \times \#tiles$. Then, the total cycles to load outputs into output buffers ($OutBuf$) are summed up across all the convolution layers as

$$\#cycles_WrOutBuf = \sum_{L=1}^{\#CONVs} word_OutBuf[L] \times \#tiles[L]. \quad (35)$$

The size (bit) of results written into the output buffers is

$$bit_WrOutBuf = \#cycles_WrOutBuf \times \#OutBuf \times Pox \times bit_Px. \quad (36)$$

Since each output is written into and read from the output buffers only once, the size (bit) of data read from output buffers ($bit_RdOutBuf$) by DMA equals to $bit_WrOutBuf$.

VIII. EXPERIMENTS AND ANALYSIS

In this section, the proposed performance model is used to explore the design space by tuning the key design variables, e.g., unrolling and tiling sizes, and DRAM bandwidth and accelerator frequency, to identify the performance bottleneck and obtain the optimal design configurations.

A. Design Space Exploration of Tiling Variables

The loop tiling strategy determines how much data of each layer is buffered, which affects the buffer capacity requirement, the number of DRAM accesses, and the accelerator performance. Although we have fixed $Tkx = Nkx$, $Tky = Nky$, $Tif = Nif$, and $Tox = Nox$ as mentioned in Section II-D, the

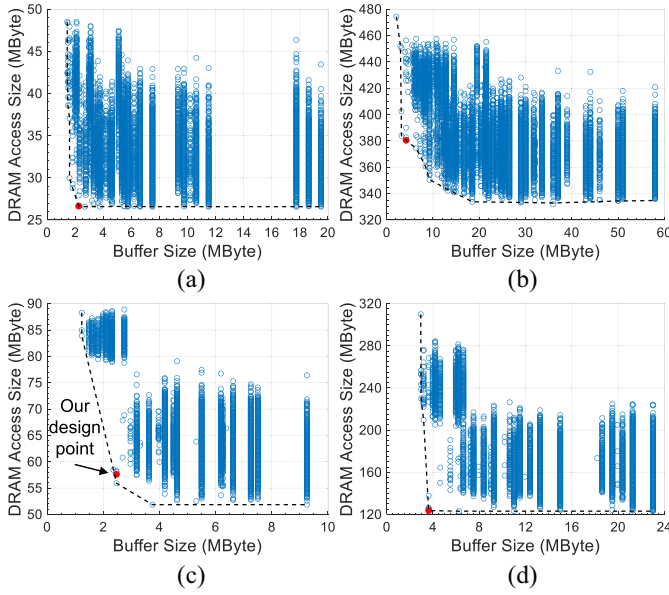


Fig. 9. Tiling variables (T_{oy} and T_{of}) are swept to explore the relationship between the size of DRAM accesses and the total input/weight/output buffer size requirement, where $P_{ox} \times P_{oy} \times P_{of} = 7 \times 7 \times 32$ with 16-bit data. (a) NiN. (b) VGG-16. (c) GoogLeNet. (d) ResNet-50.

remaining two tiling variables T_{oy} and T_{of} still give us a huge design space as mentioned in [11]. For example, VGG-16 has 13 convolution layers, and there are $13 \times 2 = 26$ tiling variables and each variable can have 4 or more candidate values determined by N_{oy}/P_{oy} or N_{of}/P_{of} , then the total number of T_{oy} and T_{of} choices is roughly $4^{26} = 4.5 \times 10^{15}$, which results in an enormous solution space that cannot be enumerated. Therefore, we randomly sample 30 000 tiling configurations for different CNN algorithms to explore their impact on the memory access and performance as in Figs. 9–11, where we set loop unrolling variables as $P_{ox} \times P_{oy} \times P_{of} = 7 \times 7 \times 32$.

The relationship between tiling variables and the number of DRAM accesses is investigated in Fig. 9 with 16-bit data. The total convolution DRAM access size is computed by

$$\text{byte_DRAM} = \sum_{L=1}^{\#CONVs} (\text{byte_RdPx} \cdot \#tiles_In + \text{byte_RdWt} \times \#tiles_Wt + \text{byte_WrPx} \cdot \#tiles) \quad (37)$$

where the right-hand side variables are computed by (11), (12), (14), (29), and (30). The DRAM accesses of other layers are also included in Fig. 9. One circle in Fig. 9 represents one design point of the tiling variables T_{oy} and T_{of} . Since the buffer size is determined by the layer with the maximum tiling size, there could be multiple different tiling configurations in other layers leading to the same buffer size. The buffer size in Fig. 9 includes input/weight/output buffers, which equals to $\max(\text{bit_InBuf}) + \max(\text{bit_WtBuf}) + \max(\text{bit_OutBuf})$ from (21), (23), and (25). With the increase of tiling and buffer sizes, the number of DRAM accesses is decreasing as shown by the dashed line in Fig. 9. After the buffer size is increased to be large enough, we can achieve the

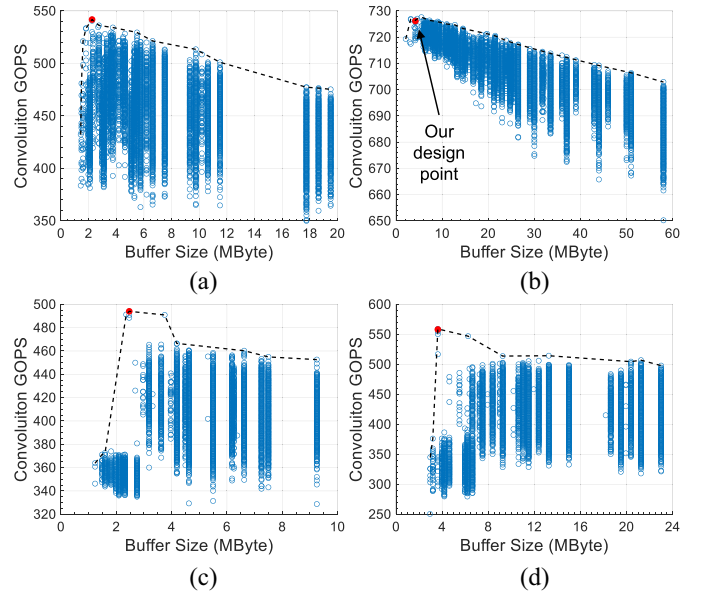


Fig. 10. Tiling variables (T_{oy} and T_{of}) are swept to explore the relationship between the convolution throughputs and the total input/weight/output buffer size requirement, where $P_{ox} \times P_{oy} \times P_{of} = 7 \times 7 \times 32$, $MHz_Accelerator = 240$, $BW_DRAM = 14.4$ GB/s. (a) NiN. (b) VGG-16. (c) GoogLeNet. (d) ResNet-50.

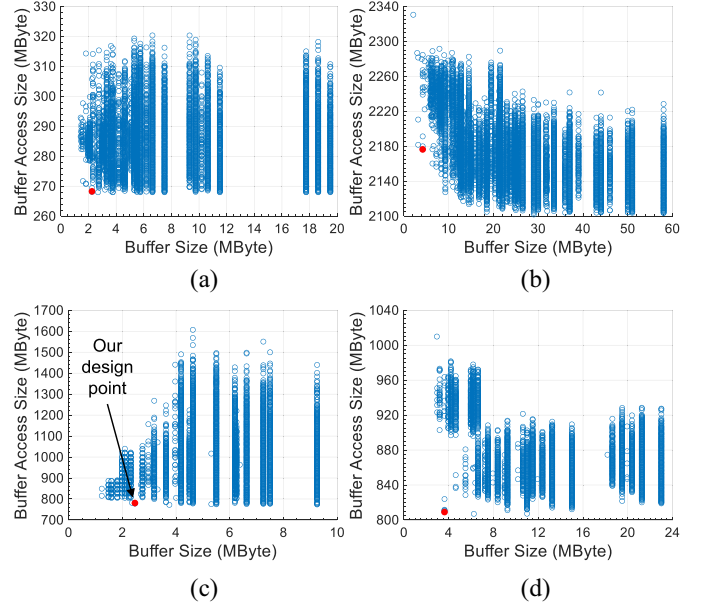


Fig. 11. Tiling variables (T_{oy} and T_{of}) are swept to explore the relationship between the size of on-chip buffer accesses and the size requirement of buffers, where $P_{ox} \times P_{oy} \times P_{of} = 7 \times 7 \times 32$. (a) NiN. (b) VGG-16. (c) GoogLeNet. (d) ResNet-50.

minimum DRAM accesses. The red dot in Fig. 9 is our optimal design choice of T_{oy} and T_{of} that balances the buffer size requirement and the number of DRAM accesses.

Fig. 10 shows the relationship between tiling sizes and the convolution throughputs, where the accelerator operating frequency is 240 MHz and the DRAM bandwidth is 14.4 GB/s. The throughput is computed by $\#operations/delay$, where $\#operations = 2$ Nm including both multiply and addition, and $delay$ is the sum of ms_1CV over all the convolution

layers. If the tiling or buffer size is too small, the number of DRAM access and the associated latency is significantly increased, which degrades the throughput. If the tiling size is too large or there is only one tile in one layer, the DRAM access latency cannot be well overlapped with the computation delay as mentioned in Section V-B, which results in lower throughput. This trend is shown by the dashed line in Fig. 10. The dashed lines of GoogLeNet and ResNet-50 are not as smooth as those of NiN and VGG-16. It is mainly because GoogLeNet and ResNet-50 have more layers resulting in much larger design space, which makes it more difficult to cover all the design choices through random sampling. The red dots in Fig. 10 are our design choices of *Toy* and *Tof*, which are the same in Fig. 9, to achieve the best throughputs.

Fig. 11 shows the relationship between tiling sizes and the number of on-chip buffer accesses for different CNN algorithms, which include both read and write operations of input/weight/output buffers of all the layers in a given CNN algorithm. Based on our acceleration strategy [11], the partial sums are accumulated inside the MAC units, which do not involve buffer access. The estimation of the number of on-chip buffer accesses is discussed in Section VII. Our design choices of *Toy* and *Tof* shown by red dots in Fig. 11 can achieve close to the optimal number of buffer accesses while having best throughputs and low level of DRAM accesses.

B. Design Space Exploration for Performance

As convolution dominates the CNN operations [2]–[4], [20], we focus on the design space exploration of convolution throughputs. The convolution throughput is affected by several factors, namely the accelerator operating frequency, external memory bandwidth, and the loop unrolling variables, these are explored in Fig. 12 using GoogLeNet as an example. With a small number of MAC units and high DRAM bandwidth (BW_DRAM) as shown in Fig. 12(a), the accelerator throughput is mainly bounded by computation, and thus the throughput is almost linearly increasing with the frequency when $BW_DRAM > 12.8$ GB/s. If the DRAM bandwidth is too low, e.g., 3.2 GB/s, the design is more likely to be memory bounded and the throughput stops increasing with the frequency. With more MAC units and higher frequency, the throughputs are tend to increase, as shown in Fig. 12, until the design touches the memory roof which is illustrated in Fig. 13.

The memory roof throughput [7] in Fig. 13 is the maximum achievable throughput under a certain external memory bandwidth and it is defined as

$$\begin{aligned} DRAM_roof(GOPS) &= \frac{\#operations(GOP)}{DRAM_delay(s)} \\ &= \frac{\#operations(GOP)}{\#data(GByte)} BW_Memory(GB/s) \end{aligned} \quad (38)$$

where $\#data$ is the data size of DRAM accesses. Since the computation-to-communication (CTC) ratio, i.e., $\#operations/\#data$, is a constant under a certain tiling setting, $DRAM_roof$ is directly proportional to BW_Memory . With the same setting of BW_Memory for GoogLeNet and

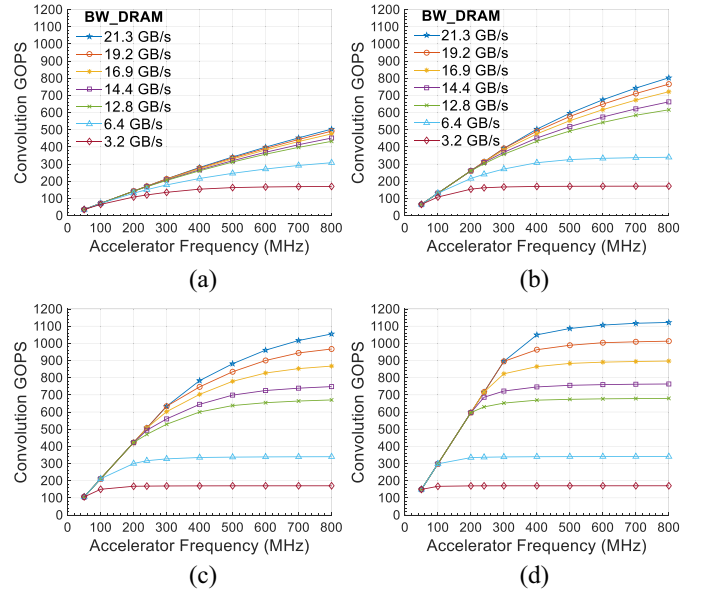


Fig. 12. Convolution throughput is affected by the accelerator operating frequency, DRAM bandwidth, and the number of MAC units. GoogLeNet is shown as an example here. (a) $Pox = 7, Poy = 7$, and $Pof = 8$. (b) $Pox = 7, Poy = 7$, and $Pof = 16$. (c) $Pox = 7, Poy = 7$, and $Pof = 32$. (d) $Pox = 14, Poy = 7$, and $Pof = 32$.

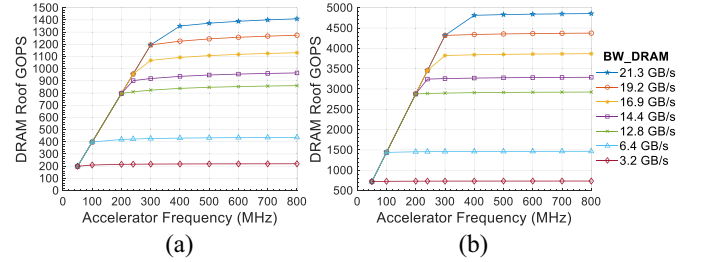


Fig. 13. External memory roof throughput ($DRAM_roof$) is the maximum achievable throughput under a certain memory bandwidth. (a) GoogLeNet. (b) VGG-16.

VGG-16, the shape of the curves in Fig. 13(a) and (b) is similar. Since VGG-16 has a higher CTC, its memory roof throughput is much higher than GoogLeNet in Fig. 13. As discussed in Section IV-B, the memory bandwidth (BW_Memory) is bounded by both the DRAM controller (BW_DRAM) and DMA (BW_DMA). At low frequency, BW_Memory is limited by BW_DMA , and $DRAM_roof$ is linearly increasing with the increase of frequency as in Fig. 13. After BW_DMA is larger than BW_DRAM , BW_Memory is limited by BW_DRAM instead, and $DRAM_roof$ stops growing with the frequency. The saturated throughputs in Fig. 12 are lower than $DRAM_roof$ in Fig. 13, which is mainly because there are redundant DRAM transfers and the computation delay is not fully overlapped with DRAM latency.

C. Performance Model Validation

Fig. 14 shows the comparison of throughput and latency between the performance model and the on-board test results on Arria 10 and Stratix 10 with different number of MAC units, where both pixels and weights are 16-bit fixed point

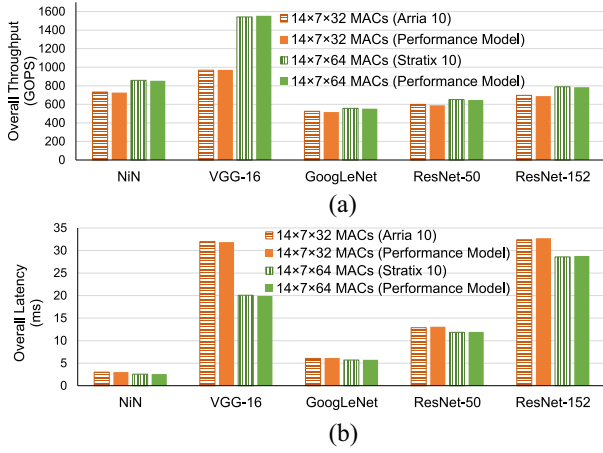


Fig. 14. Performance model results are compared with on-board test results of Arria 10 and Stratix 10 on overall (a) throughput and (b) latency.

data. The differences between the estimation and on-board results are within 3%, which are mainly due to the DRAM transfer latency mismatch, minor layers (e.g., average pooling), and some pipeline stages in the real implementation. The compilation of our FPGA design using Quartus Pro 17.1 on 16-core Intel Xeon CPU E5-2650 v3 normally takes 6–8 h, while the performance model running on Laptop Intel Core i7-7500U CPU using MATLAB takes about 1–5 s per design.

D. Related Works

Several related works have used performance model to optimize the memory access and computation pattern of their proposed architecture and dataflow. Suda *et al.* [8] implemented convolution as matrix multiplication and uses a performance model to optimize the design. However, the execution time in [8] only counts computation time without considering the DRAM transfer latency. If the design becomes memory bounded, the model in [8] cannot properly predict the overall latency, which results in the estimation discrepancy of FC layers with high computation parallelism. The proposed systolic array architecture in [10] is also optimized through a performance model. The overall throughput is simply computed by the minimum of the computation throughput and DRAM transfer throughput, where the overlap efficiency of computation and data transfer is not considered. The fine-grained tile-level data accesses of DRAM and buffers are not explored in [10]. The buffer and DRAM accesses are modeled in [18] to explore different data reuse patterns by changing the tiling strategy and computation order. Only coarse-grained modeling of the convolution memory access is analyzed without considering the DRAM bandwidth utilization and the detailed data storage patterns in buffers and DRAM. The proposed hybrid data reuse in [18] is similar to our tiling strategy that different layers can use different tiling sizes to either reuse weights or pixels to minimize the DRAM access. In this paper, the relationship between the overall DRAM access and the total buffer size is also investigated. The power of data movement in different hierarchy, e.g., DRAM, buffer, and PE

array, is analytically modeled in [19] to compare the energy efficiency of different dataflow. However, the power is not quantitatively formulated with the design variables in [19], and the performance of the accelerator is not modeled.

IX. FURTHER IMPROVEMENT OPPORTUNITIES

In this section, we use the proposed performance model to evaluate the opportunities that may further enhance the performance of the accelerator by improving the efficiency of DRAM transactions and DSP utilization.

A. Improving DRAM Bandwidth Utilization

To simplify the control logic of data bus from DMA to input buffers, different feature map rows are aligned in different addresses in our current design. By this means, if the number of pixels in one row is smaller than $\lfloor bit_DMA/bit_Px \rfloor$, the successive row directly starts from the next address instead of continuously using the same address resulting in the waste of DMA datawidth. For example, with $bit_Px = 16$, one address can accommodate $512/16 = 32$ pixels, if the width of the feature map is $Nix = 14$, then the actual number of pixels of one row read from DRAM in (11) is $Tix = 32$, where $32 - 14 = 20$ data are redundant. Some CNN models, e.g., GoogLeNet and ResNet, have a lot of convolution layers with small Nix , e.g., 7 or 14, then their throughputs are significantly affected by the inefficient utilization of DMA datawidth.

To improve the DRAM bandwidth utilization, one method is to store multiple rows in one DMA address, which involves the modifications of control logic and extra data paths from DMA to input buffers. The other method is to keep the data aligned, but narrow the bit width of the data bus between DMA and input buffers. To attain the same data transfer rate, higher frequency is needed, and asynchronous FIFO may be used. In the performance model, we reduce bit_DMA to be 256 and 128 and increase their corresponding frequency of the data bus to predict the throughput improvements. In Fig. 15, our current design (DMA 512-bit) serves as the baseline with data aligned, and bit_DMA is set to be 256 or 128, which has the same effect as supporting two or four rows in one address with $bit_DMA = 512$, respectively. Fig. 15 shows that NiN, GoogLeNet, and ResNet can benefit a lot from decreasing the DMA bit width, mainly because they have many layers with small Nix and the layers with small Nix are memory bounded. On the contrary, VGG-16 cannot benefit from higher DRAM bandwidth utilization as the design is still computationally bounded. Based on the prediction, it is compelling to improve our design for higher DRAM bandwidth utilization.

B. Merging the First Layers

In GoogLeNet and ResNet, there are multiple parallel branches of layers, and the first layer of each branch reads input pixels from the same precedent layer. If these convolution layers also have the same kernel size and stride, they can be merged into one layer along the output feature map dimension (Nof). By this means, the input pixels can be shared by the first layers of different branches and only need to be read

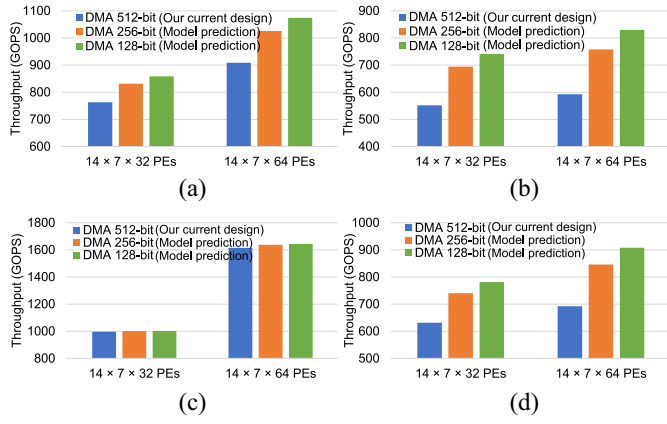


Fig. 15. Performance model predicts that the throughput will be improved by increasing the DRAM bandwidth utilization, which is achieved by decreasing the DMA bit width to reduce the redundant DRAM accesses. (a) NiN. (b) GoogLeNet. (c) VGG-16. (d) ResNet-50.

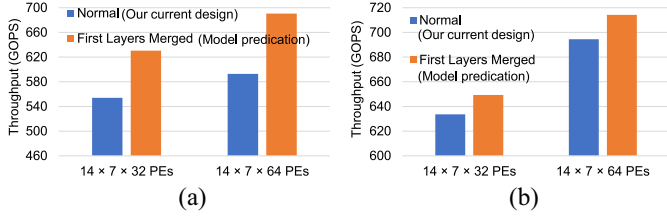


Fig. 16. Performance model predicts that the throughput will be improved by merging the first layers of different parallel branches, which read from the same precedent layer, to eliminate the repeated DRAM access, where “Normal” denotes our current design as baseline. (a) GoogLeNet. (b) ResNet-50.

from DRAM once, as proposed in [13]. We change the corresponding settings of our performance model, e.g., *byte_RdPx* in (11), to estimate the effect of eliminating the repeated DRAM accesses of the precedent layer as shown in Fig. 16. Since GoogLeNet and ResNet are already memory bounded in our current design, reducing the DRAM access can considerably improve the throughputs. The required modifications of our current design to merge the first layers involve changing the control logic and the descriptors of DMA transactions, and there is no significant overhead of additional hardware resources.

C. Improving PE Efficiency

Due to the highly varying dimensions of different convolution layers in a given CNN model, it is a challenge task to efficiently distribute workloads across PEs, or we need to make loop dimensions (N^*) divisible by their corresponding unrolling variables (P^*). In [21] and [22], an adaptive parallelism scheme is proposed to dynamically adjust the mapping of operations on different PEs, or the unrolling variables can be changed for each layer to maximize PE utilization. This requires the ability to dynamically redirect data flow from buffers to PEs, which may need complex control logic, incur penalty of additional resources, and aggravate the burden on timing closure.

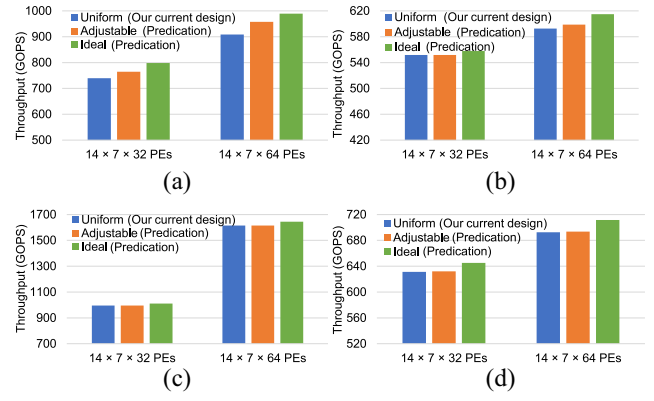


Fig. 17. Uniform: our current design as baseline with uniform PE mapping; Adjustable: dynamically adjust the unrolling variables for different layers to improve PE utilization; Ideal: force PE utilization to be 100%. (a) NiN. (b) GoogLeNet. (c) VGG-16. (d) ResNet-50.

Instead of using uniform PE mapping and unrolling variables in the current design, we adjust unrolling variables ($P_{ox} \cdot P_{oy} \cdot P_{of}$) for different layers to achieve better PE utilization in the performance model as shown by “Adjustable” in Fig. 17. We also force PE utilization to be 100% by removing the ceiling functions in (2), which is denoted by “Ideal” in Fig. 17. However, the throughput improvements from adjustable unrolling strategy are very limited ($<10\%$) for our design, mainly because: 1) the $Nox \cdot Noy \cdot Nof$ dimensions of most layers have already been able to provide large enough parallelism for our uniform unrolling strategy and 2) most of our layers are memory bounded and the reduction of computation latency has little effect on the throughput. Considering the large amount of necessary design efforts for adjustable PE mapping and low expected improvements, we surmise it is not a primary task in our future work to adopt this technique.

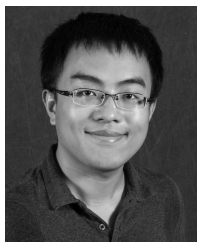
X. CONCLUSION

In this paper, a high-level performance model is proposed to estimate the key specifications, e.g., throughput, of FPGA accelerators for CNN inference, which enables the design space exploration to identify performance bottleneck in the early development phase. The design strategy and resource costs are formulated using the design variables of loop unrolling and tiling. The proposed performance model is validated for a specific acceleration strategy across a variety of CNN algorithms comparing with on-board test results on two different FPGAs.

REFERENCES

- [1] O. Russakovsky *et al.*, “ImageNet large scale visual recognition challenge,” *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. Conf. Neural Inf. Process. Syst. (NIPS)*, 2012, pp. 1097–1105.
- [3] K. Simonyan and A. Zisserman. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. [Online]. Available: <http://arxiv.org/abs/1409.1556>
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

- [5] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 1–9.
- [6] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. Conf. Artif. Intell. (AAAI)*, Feb. 2017, pp. 4278–4284.
- [7] C. Zhang *et al.*, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, Monterey, CA, USA, 2015, pp. 161–170.
- [8] N. Suda *et al.*, "Throughput-optimized OpenCL-based FPGA accelerator for large-scale convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, Monterey, CA, USA, 2016, pp. 16–25.
- [9] C. Zhang, Z. Fang, P. Zhou, P. Pan, and J. Cong, "Caffeine: Towards uniformed representation and acceleration for deep convolutional neural networks," in *Proc. Int. Conf. Comput.-Aided Design (ICCAD)*, Austin, TX, USA, Nov. 2016, pp. 1–8.
- [10] X. Wei *et al.*, "Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs," in *Proc. Design Autom. Conf. (DAC)*, Austin, TX, USA, 2017, pp. 1–6.
- [11] Y. Ma, Y. Cao, S. Vrudhula, and J.-S. Seo, "Optimizing the convolution operation to accelerate deep neural networks on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 7, pp. 1354–1367, Jul. 2018.
- [12] H. Zeng, R. Chen, C. Zhang, and V. K. Prasanna, "A framework for generating high throughput CNN implementations on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, Monterey, CA, USA, Feb. 2018, pp. 117–126.
- [13] X. Lin *et al.*, "LCP: A layer clusters paralleling mapping method for accelerating inception and residual networks on FPGA," in *Proc. Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.
- [14] K. Pavel and S. David, "Algorithms for efficient computation of convolution," in *Design and Architectures for Digital Signal Processing*. New York, NY, USA: IntechOpen, Jan. 2013. doi: [10.5772/51942](https://doi.org/10.5772/51942).
- [15] J. Yu *et al.*, "Real-time object detection towards high power efficiency," in *Proc. Design Autom. Test Eur. Conf. Exhibit. (DATE)*, Dresden, Germany, Mar. 2018, pp. 704–708.
- [16] C. Zhang and V. K. Prasanna, "Frequency domain acceleration of convolutional neural networks on CPU-FPGA shared memory system," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, Monterey, CA, USA, 2017, pp. 35–44.
- [17] Y. Ma, Y. Cao, S. Vrudhula, and J.-S. Seo, "An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks," in *Proc. Int. Conf. Field Program. Logic Appl. (FPL)*, Ghent, Belgium, Sep. 2017, pp. 1–8.
- [18] F. Tu *et al.*, "Deep convolutional neural network architecture with reconfigurable computation patterns," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 8, pp. 2220–2233, Aug. 2017.
- [19] Y.-H. Chen, J. S. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA)*, Seoul, South Korea, Jun. 2016, pp. 367–379.
- [20] M. Lin, Q. Chen, and S. Yan, "Network in network," *CoRR*, vol. abs/1312.4400, 2013. [Online]. Available: <http://arxiv.org/abs/1312.4400>
- [21] L. Song *et al.*, "C-Brain: A deep learning accelerator that tames the diversity of CNNs through adaptive data-level parallelization," in *Proc. Design Autom. Conf. (DAC)*, Austin, TX, USA, Jun. 2016, pp. 1–6.
- [22] M. Putic *et al.*, "Dyhard-DNN: Even more DNN acceleration with dynamic hardware reconfiguration," in *Proc. Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.



Yufei Ma (S'16) received the B.S. degree in information engineering from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2011, and the M.S.E. degree in electrical engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2013. He is currently pursuing the Ph.D. degree with Arizona State University, Tempe, AZ, USA.

His current research interests include high-performance hardware acceleration of deep learning algorithms on digital application-specified integrated

circuit and field-programmable gate array.



Yu Cao (S'99–M'02–SM'09–F'17) received the B.S. degree in physics from Peking University, Beijing, China, in 1996, the M.A. degree in biophysics, and the Ph.D. degree in electrical engineering from the University of California at Berkeley, Berkeley, CA, USA, in 1999 and 2002, respectively.

He was a Summer Intern with Hewlett-Packard Labs, Palo Alto, CA, USA, in 2000, and with IBM Microelectronics Division, East Fishkill, NY, USA, in 2001. He was a Post-Doctoral Researcher with the Berkeley Wireless Research Center, Berkeley. He is currently a Professor of electrical engineering with Arizona State University, Tempe, AZ, USA. He has published numerous articles and two books on nano-CMOS modeling and physical design. His current research interests include physical modeling of nanoscale technologies, design solutions for variability and reliability, reliable integration of post-silicon technologies, and hardware design for on-chip learning.

Dr. Cao was a recipient of the 2012 Best Paper Award at IEEE Computer Society Annual Symposium on Very Large Scale Integration, the 2010, 2012, 2013, 2015, and 2016 Top 5% Teaching Award at Schools of Engineering, Arizona State University, the 2009 ACM SIGDA Outstanding New Faculty Award, the 2009 Promotion and Tenure Faculty Exemplar at Arizona State University, the 2008 Chunhui Award for Outstanding Oversea Chinese Scholars, the 2007 Best Paper Award at the International Symposium on Low Power Electronics and Design, the 2006 NSF CAREER Award, the 2006 and 2007 IBM Faculty Award, the 2004 Best Paper Award at International Symposium on Quality Electronic Design, and the 2000 Beatrice Winner Award at International Solid-State Circuits Conference. He has served as Associate Editor for the IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, and served on the technical program committees of many conferences. He was a Distinguished Lecturer of the IEEE Circuits and Systems Society in 2009.



Sarma Vrudhula (M'85–SM'02–F'16) received the B.Math. degree from the University of Waterloo, Waterloo, ON, Canada, and the M.S.E.E. and Ph.D. degrees in electrical and computer engineering from the University of Southern California, Los Angeles, CA, USA.

He was a Professor with the ECE Department, University of Arizona, Tucson, AZ, USA, and was on the Faculty of the EE-Systems Department, University of Southern California. He was also the Founding Director of the NSF Center for Low Power Electronics, University of Arizona, Tempe, AZ, USA, where he is a Professor of Computer Science and Engineering, and the Director of the NSF IUCRC Center for Embedded Systems. More recently, he has been investigating non-conventional methods for implementing logic, including technology mapping with threshold logic circuits, the implementation of threshold logic using resistive memory devices, and the design and optimization of nonvolatile logic. His current research interests include design automation and computer aided design for digital integrated circuit and systems, focusing on low power circuit design, and energy management of circuits and systems, energy optimization of battery-powered computing systems, including smartphones, wireless sensor networks, and IoT systems that rely on energy harvesting, system level dynamic power and thermal management of multicore processors and system-on-chip, statistical methods for the analysis of process variations, statistical optimization of performance, power and leakage, new circuit architectures of threshold logic circuits for the design of application-specified integrated circuits, and field-programmable gate array.



Jae-Sun Seo (S'04–M'10–SM'17) received the B.S. degree in electrical engineering from Seoul National University, Seoul, South Korea, in 2001, and the M.S. and Ph.D. degrees in electrical engineering from the University of Michigan, Ann Arbor, MI, USA, in 2006 and 2010, respectively.

From 2010 to 2013, he was with IBM T. J. Watson Research Center, Yorktown Heights, NY, USA, where he researched on cognitive computing chips under the DARPA SynAPSE Project and energy-efficient integrated circuits for high-performance processors. In 2014, he joined the School of Electrical, Computer, and Energy Engineering, Arizona State University, Tempe, AZ, USA, as an Assistant Professor. In 2015, he was with the Intel Circuits Research Laboratory as a Visiting Faculty Member. His current research interests include efficient hardware design of machine learning and neuromorphic algorithms and integrated power management.

Dr. Seo was a recipient of the Samsung Scholarship from 2004 to 2009, the IBM Outstanding Technical Achievement Award in 2012, and the NSF CAREER Award in 2017.